

統合開発環境(IDE)と バージョン管理について

吉田稔(エコ棟704, mino@is.tokushima-u.ac.jp)

大きなプログラムを複数人で作る

□ 大規模プロジェクト

□ ファイルが多くなる

- 沢山のプログラムファイル、画像ファイル、音楽ファイル、etc.

□ プログラムも長くなる

- どこにどのルーチンがあるかわからない！

□ ⇒IDEによるプログラミング補助

□ 複数人による開発

□ 誰が何を開発・修正したのか混乱

□ ⇒バージョン管理システム

統合開発環境とは？

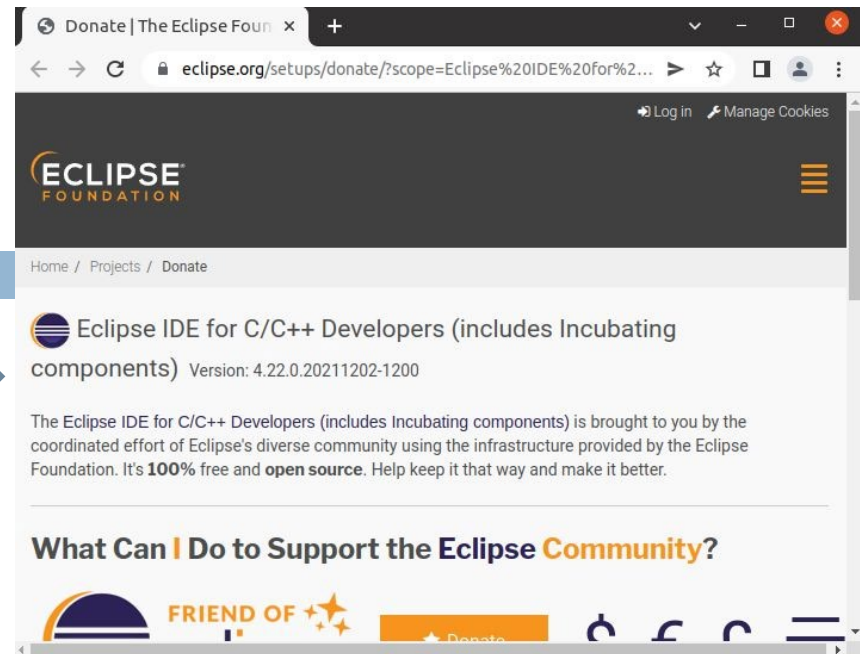
- IDE (Integrated Development Environment)の略
- 特徴
 - ▣ プロジェクト管理
ソースファイル、ヘッダ、アイコンなどを1まとまりで管理できる
 - ▣ バージョン管理
反管理を行うことより、チーム開発をサポートできる。
 - ▣ GUIの作成支援
ビジュアルにGUIを作ることができる。
 - ▣ コーディングの補助：
{}の対応や予約語の記述ミス等を防ぐことができる。
 - ▣ ビルド&デバッグの補助
コンパイラやリンカと連携して、簡単に実行できる
デバッガと連携して、ブレイクポイントを置いたりできる。

IDEの例

- Eclipse
- CodeWarrior
- Xcode (Apple)
- Visual studio (Microsoft)
- など

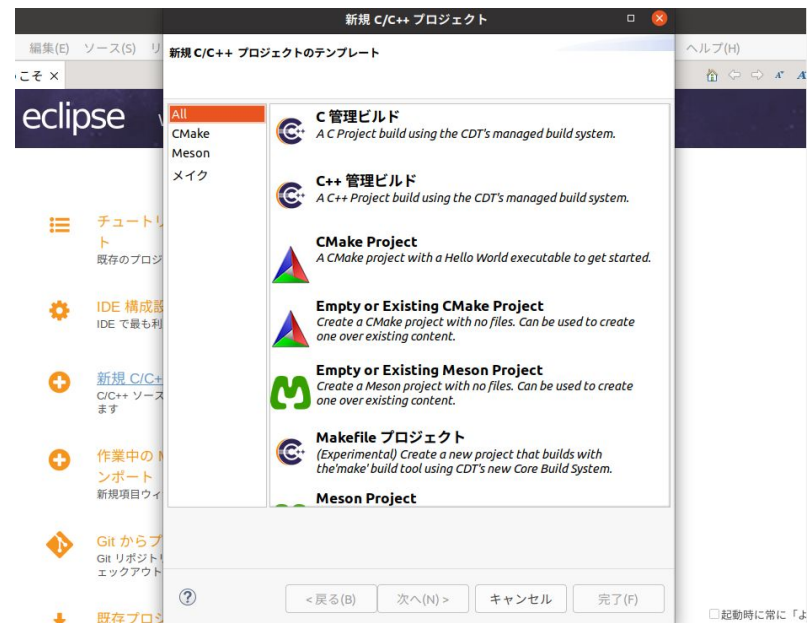
Eclipseを試してみよう

- 端末でeclipseと入力して実行
 - ▣ ワークスペースを選択。デフォルトでOK
 - ▣ 「新規C/C++プロジェクトの作成」を選択
 - 「C管理ビルド」を選択
 - 「Hello World ANSI Cプロジェクト」を選択
 - Hello Worldプロジェクトを選択すると、最初からHello Worldのプログラムがセットされたプロジェクトが作成される。
 - ▣ トンカチのアイコンを押すとビルドする。
 - ▣ 実行は再生のアイコンを押す。



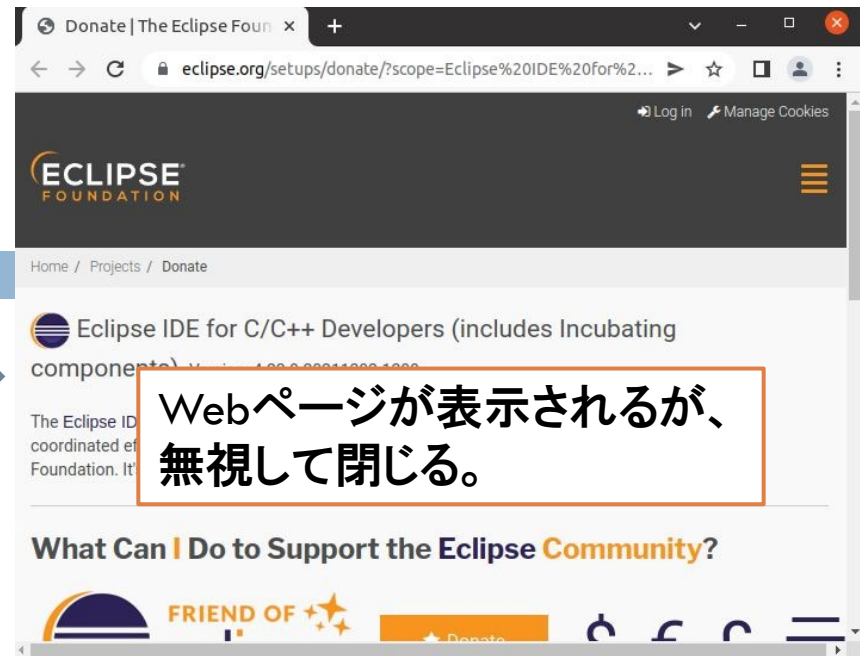
- チュートリアル: 既存のプロジェクトをインポート
既存のプロジェクトをインポートするガイド付きウォークスルー
- IDE 構成設定の確認
IDE で最も利用されている設定を確認します
- 新規 C/C++ プロジェクトの作成
C/C++ ソース・コード用の新規 Eclipse プロジェクトを作成します
- 作業中の Makefile を使用してプロジェクトをインポート
新規項目ウィザードを開きます
- Git からプロジェクトをチェックアウト
Git リポジトリにホストされている Eclipse プロジェクトのチェックアウト
- 既存プロジェクトのインポート

- 概要
フィーチャーの概要
- チュートリアル
チュートリアルの実行
- サンプル
サンプルの試行
- 新機能
新機能について





デフォルトのままでよい。



Webページが表示されるが、無視して閉じる。



チュートリアル: 既存のプロジェクトをインポート
既存のプロジェクトをインポートするガイド付きワークスルー

IDE 構成設定の確認
IDE で最も利用されている設定を確認します

新規 C/C++ プロジェクトの作成
C/C++ ソース・コード用の新規 Eclipse プロジェクトを作成します

Git からプロジェクトをチェックアウト
Git リポジトリにホストされている Eclipse プロジェクトのチェックアウト

既存プロジェクトのインポート

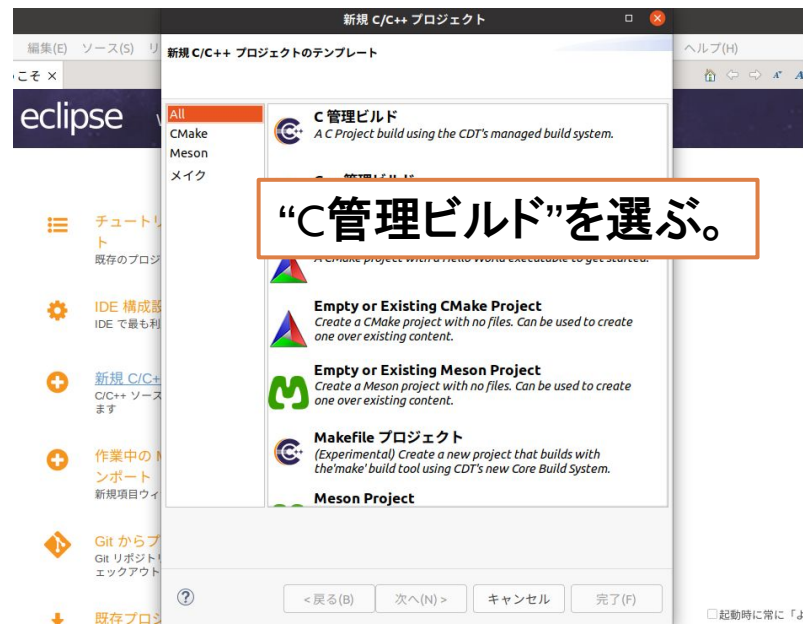
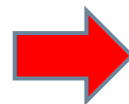
概要
フィーチャーの概要

チュートリアル
チュートリアルの実行

サンプル
サンプルの試行

「新規C/C++プロジェクト」を選ぶ。

新規項目ウィザードを開きます



“C管理ビルド”を選ぶ。

C プロジェクト

選択したタイプのCプロジェクトを作成します

プロジェクト名(P): test

☒ デフォルト・ロケーションを使用(D)

ロケーション(L): /home/vmware/eclipse-workspace/test

参照(R)...

ファイル・システムを選択(Y): デフォルト

プロジェクト・タイプ:

ツールチェーン:

- GNU Autotools
- static ライブラリー
- 共用ライブラリー
- 実行可能
 - 空のプロジェクト
 - Hello World ANSI C プロジェクト**
 - Makefile プロジェクト

Cross GCC
Linux GCC

☒ プラットフォームでサポートされる場合のみ、プロジェクト・タイプとツールチェーンを表示

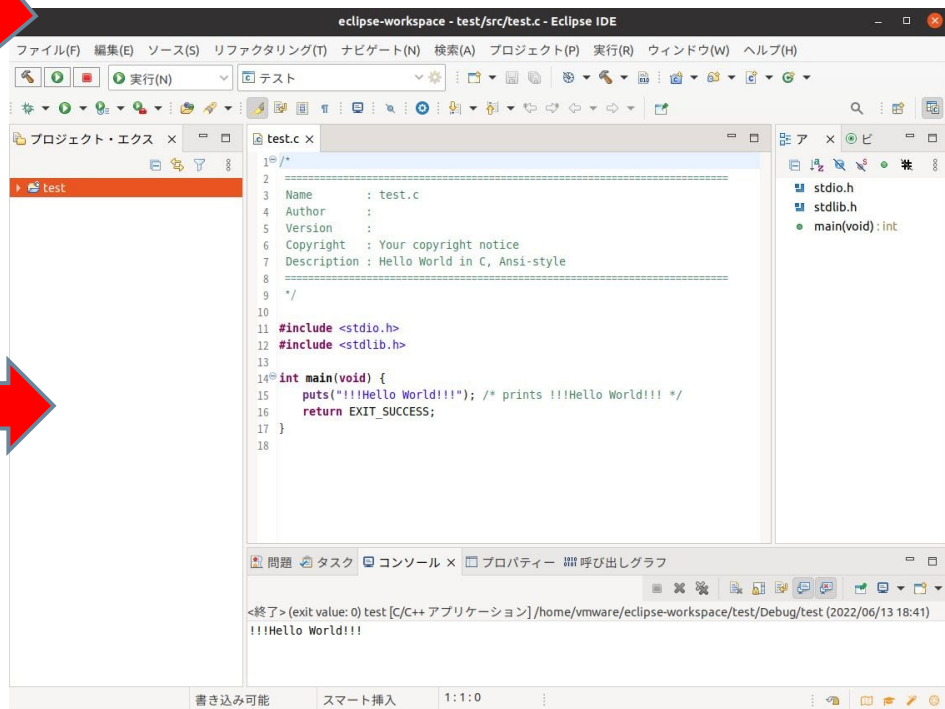
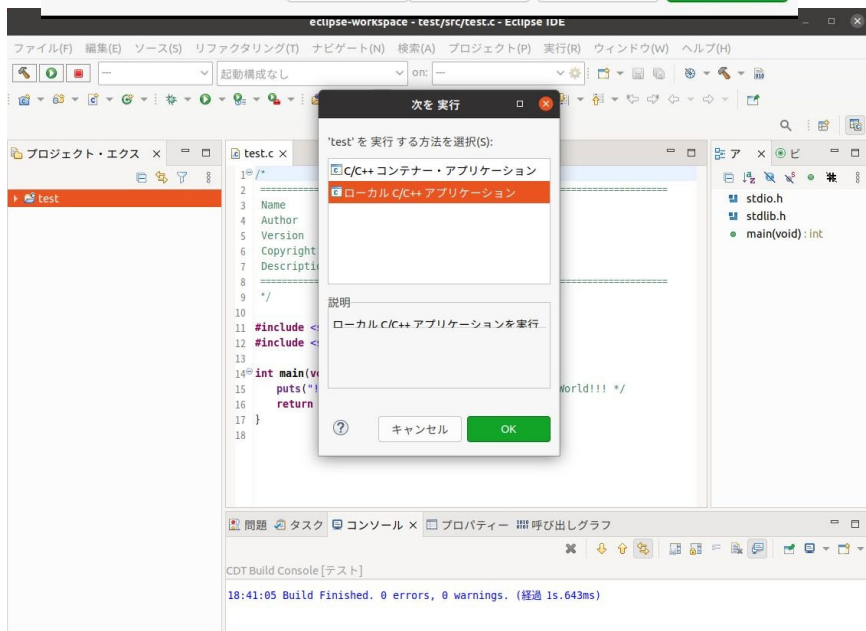
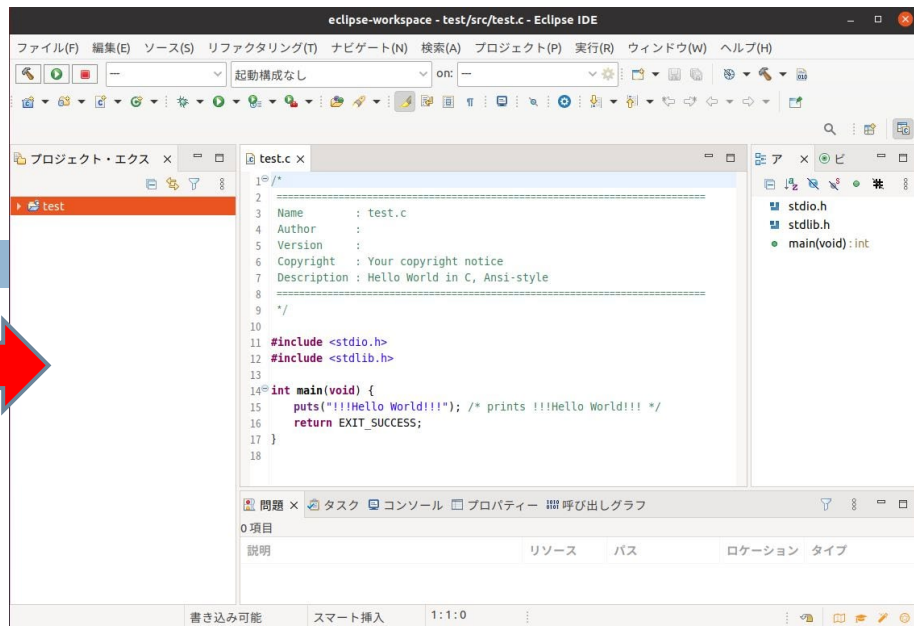


<戻る(B)

次へ(N) >

キャンセル

完了(F)



C プロジェクト

選択したタイプのCプロジェクトを作成します

プロジェクト名(P): test

☒ デフォルト・ロケーションを使用(D)

ロケーション(L): /home/vmware/eclipse-workspace/test

参照(R)...

ファイル・システムを選択(Y): デフォルト

プロジェクト・タイプ:

ツールチェーン:

- GNU Autotools
- static ライブラリー
- 共用ライブラリー
- 実行可能
 - 空のプロジェクト
 - Hello World ANSI C プロジェクト**
 - Makefile プロジェクト

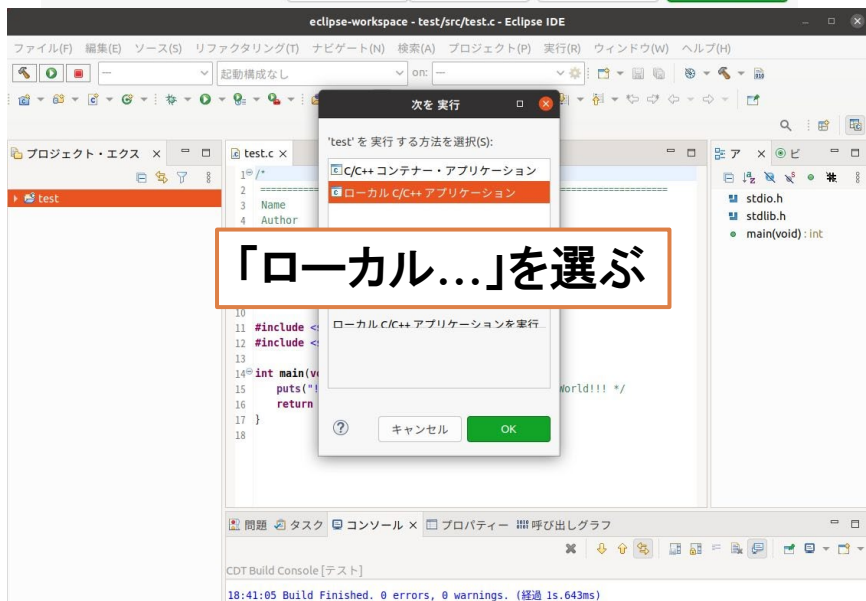
Cross GCC

Linux GCC

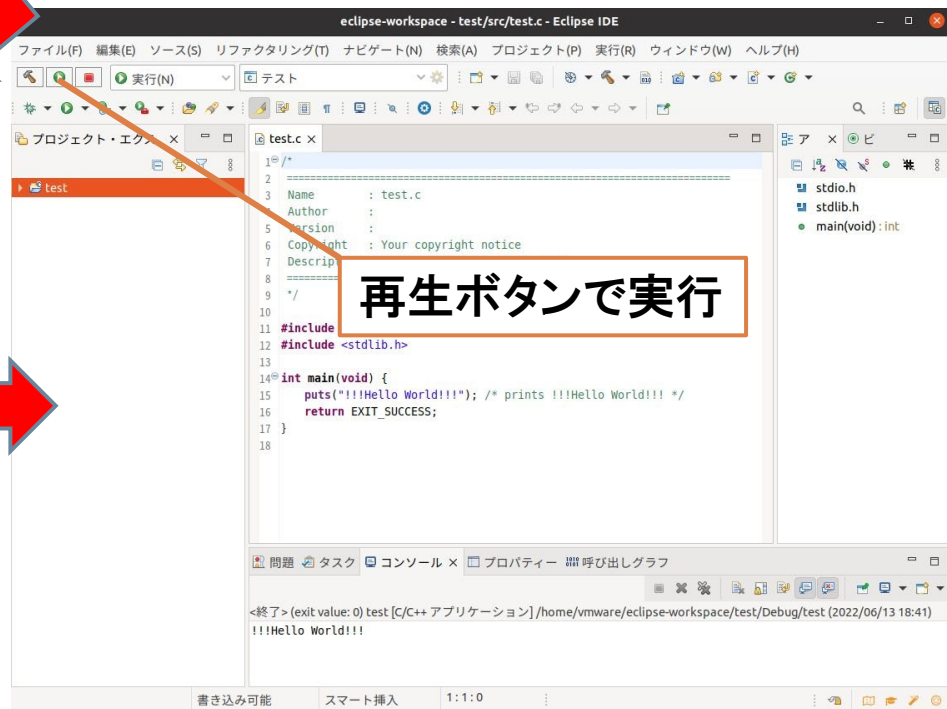
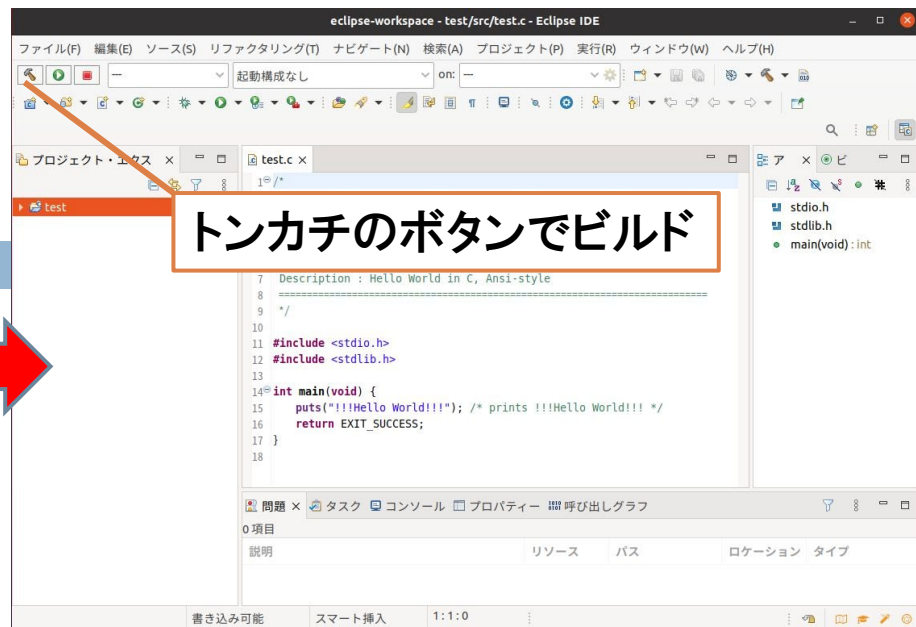
“Linux GCC”を選ぶ

Hello world ... を選ぶ

「完了」ボタンを押す



「ローカル...」を選ぶ



大きなプログラムの開発

- バージョン管理(個人でやる場合):
 - ▣ プログラムを修正する前に、前のバージョンを保存しておく。
 - main-20141016.c 等、日付や時刻をつけておく。
 - ▣ 問題が起きた時に、いつでも修正前に戻せる。
- ⇒複数人の開発では、問題が複雑になる。
 - ▣ 別の人がいつの間にかファイルを書き換えている
 - どういう修正をしたのか、(自分のやった修正じゃないので)わからない。
 - ▣ 複数の人が同時に同じファイルを書き換えようとする
 - ▣ etc...
- ⇒専用のソフトの必要性

バージョン管理とは？

- ファイルの変更履歴を管理
- メリット
 - ▣ 過去の変更履歴を残し、1カ所にまとめておける
 - ▣ 過去の任意の状態に簡単に戻せる。
 - ▣ 過去にどのような変更を行ったか、コミットログを見ることにより、把握できる。
 - ▣ グループ開発で、複数の人が同一ファイルを変更しても、衝突を検出して、調整できる。

バージョン管理システムの種類

□ 単一リポジトリ

共用のリポジトリが1つあり、全てのユーザがこれに対してコミットを行う

例: CVS, **Subversion**

□ 複数リポジトリ

クライアントがそれぞれ自分のリポジトリをもつことで各自が別々にソースコードを管理する。

例: Git, Mercurial, Bitkeeper

Subversion

- 世界的に使われているバージョン管理システムCVSを、さらに改善したもの。

Subversionの用語

- 「リポジトリ」: 共用の保存場所
- 「プロジェクト」: 作成するプログラム

リポジトリ

プロジェクト
(作業コピー)

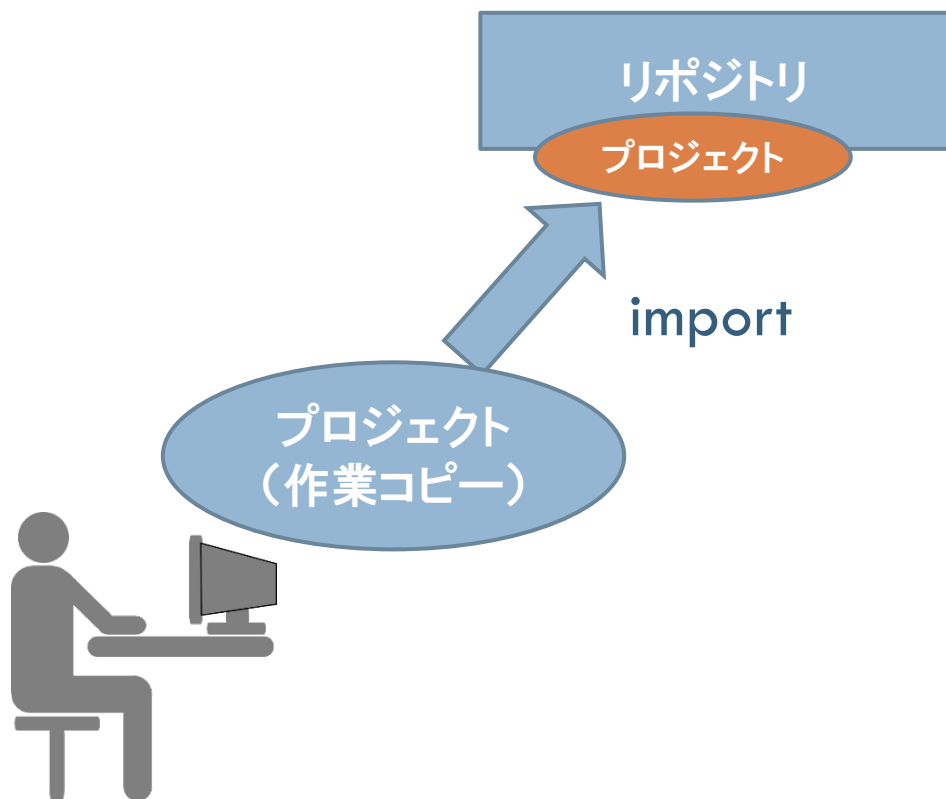


プロジェクトのコピー
(作業コピー)を
ローカルに作成し、
実際の作業は
作業コピーに対して行う。

作業が終わったら、結果を
リポジトリに登録する。

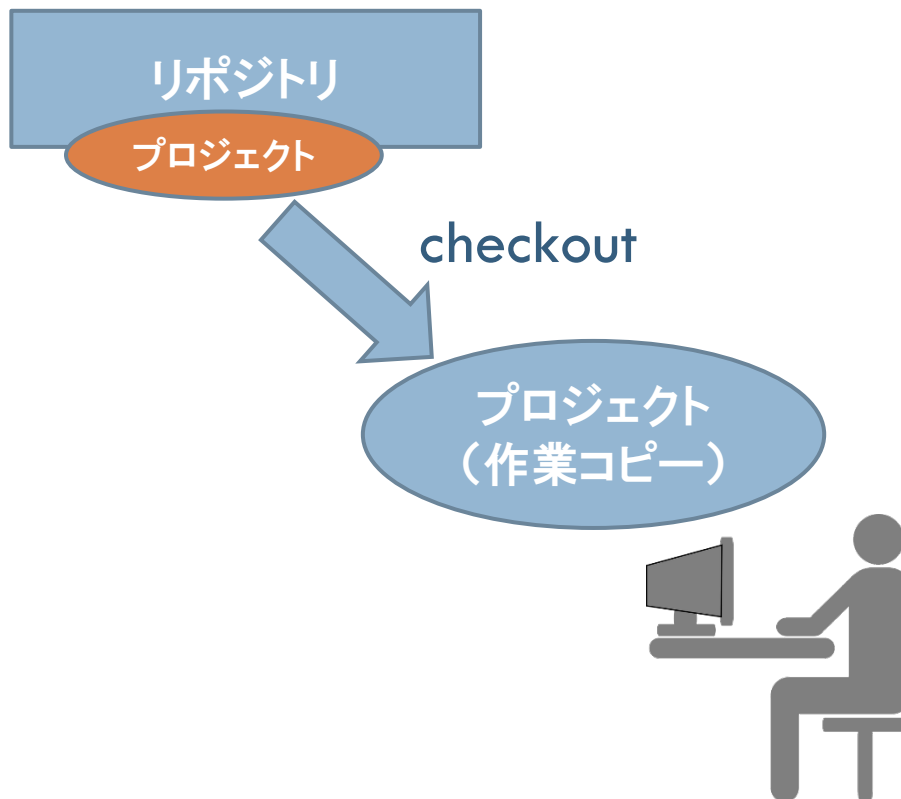
Subversionの用語

- import: 自分のプロジェクトを、リポジトリに登録する。



Subversionの用語

- checkout: 登録されたプロジェクトを、リポジトリから取り出す。



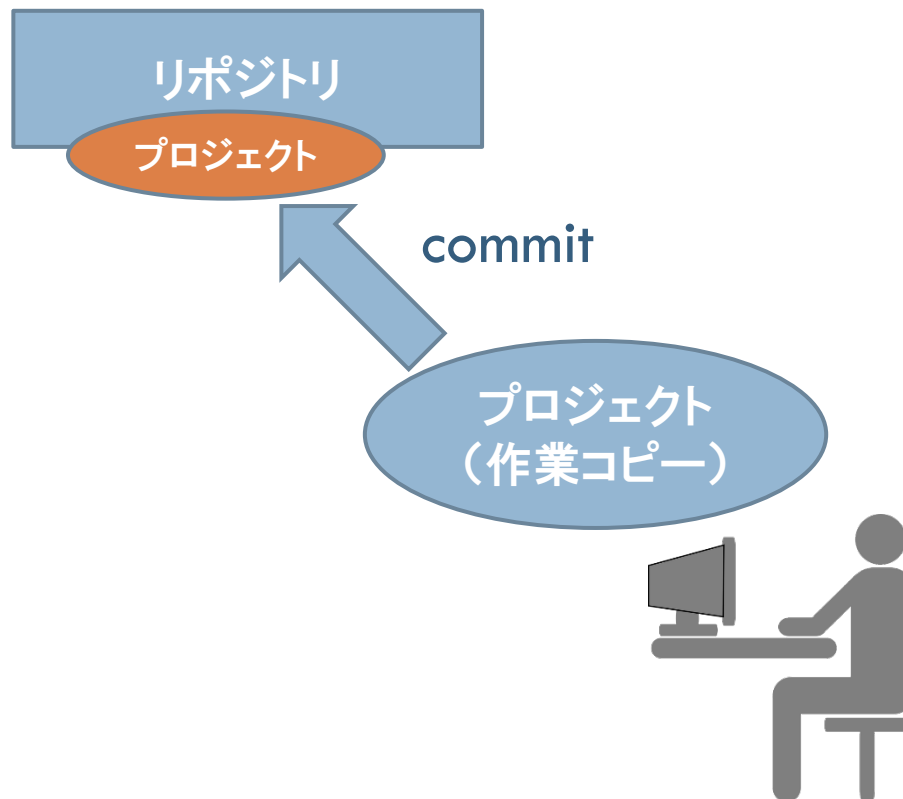
Subversionの用語

- checkoutしたプロジェクトに、いろいろ変更を加える。



Subversionの用語

- commit: 加えた変更を、リポジトリに反映する。



Subversionの使い方

- svnコマンドの使い方
 - ▣ 参考文献: <http://www.hyuki.com/techinfo/svninit.html>
- このweb pageに書かれていることと、同じことを実行してください
 - ▣ すでにインストールはされているので、「ダウンロードとインストール」は不要。
 - ▣ windows用の解説なので、一部linux用に直す必要があることに注意
 - C:¥とかは、Windowsでのディレクトリ表現なので、適宜/`usrhome/...`等に変える。
 - `type` ではなく `cat` を使う
 - `del -> rm -r`
 - `dir /a -> ls -a`
 - `file:///C:/ ...` ではなく、`file:///usrhome/...` とする
 - 等。
 - ▣ `svn checkin` のときの指定は、カレントディレクトリではなくレポジトリの場所なので注意。
 - ▣ 現在のディレクトリは、`pwd` コマンドで表示されます。

実際にやってみる：準備

- 最初のほうについて、vmware上で実行する例を以下に示します。
- ホームディレクトリにいるものと仮定(/home/vmware)
 - ▣ 各環境で、適当なものに置き換えること。
- リポジトリの位置は、URLで表記する
 - ▣ インターネットブラウザ等で、<http://www.tokushima-u...>などと表記されているやつ
 - インターネット上のファイルと、ローカルのファイルを同等に扱える表記
 - ▣ ローカルなファイルは、"file://"の後に絶対パスを表記することで表現する
 - 例：file:///home/vmware/

実際にやってみる：準備

- ホームディレクトリにいるものと仮定
(/home/vmware)
- 各環境で、適当なものに置き換えること。
- pwdで、現在いる絶対パスがわかります。
- 電算室にいる場合は、
 - /home/vmware/ の代わりに
 - /usrhome/.../c000024926/ のようになる。

実際にやってみる: 準備

□ リポジトリの初期化

- `svnadmin create /home/vmware/repos`

□ ディレクトリとファイルの作成

- `mkdir work`

- `cd work`

- `mkdir myproject`

- `cd myproject`

- `echo This is readme file. > readme.txt`

- `cat readme.txt`

実際にやってみる：登録

□ インポート

- `svn import file:///home/vmware/repos/myproject -m "Initial import."`
 - これでインポートされる(リポジトリに登録される)
- `cd ..`
- `rm -r myproject`
 - プロジェクトの作業ディレクトリを削除してみる
 - `rm -rf myproject`なら、強制的に(確認なしで)全部削除

実際にやってみる:登録

□ チェックアウト

- `svn checkout file:///home/vmware/repos/myproject myproject`
 - リポジトリに登録されているmyprojectを、現在のディレクトリにコピーする(チェックアウトする)
- `cd myproject`
- `ls`
- `cat readme.txt`
 - 先ほど登録->ローカルで削除したプロジェクトが、チェックアウトにより復活している

実際にやってみる：編集

□ ファイル編集してみる

- `echo Nice to meet you. >> readme.txt`

□ コミットしてみる

- `svn commit -m "Add a greeting."`
- `svn status`
- `svn update`

以下、同様に、参考文献の通り実行してみてください。

Tips

□ バージョン管理

- できるだけ小さい単位でコミットする
1つの関数ができたらコミット。
1つの機能ができたらコミット。
- プロジェクトに関係のあるファイルは全てチェックインする
仕様書、アイコン、発表用ファイルなどなど
ここを見れば全ての情報が手に入るようにする
- コミットするときは、必ず詳細にコメントを書く
書いていないとすぐに変更点が分からない。
- 必ずエラーがない状態でコミットする
エラーがあるとコンパイルできない。。。

□ IDE

- IDEは、完璧とは限りませんので、バックアップは、こまめにとりましょう！
- 週に一度はUSBメモリにバックアップしてください。

課題

- <http://www.hyuki.com/techinfo/svninit.html>
を参考に、適当にファイルの追加や修正を行い、update, add, status, commit等の実行結果をレポートせよ(PDF形式で提出すること)。
- 締切: 6/25(木) 12:50